

Ejemplos

Raul Gomez

2026-04-05

Exponenciación Binaria

En este notebook, mostraremos de manera práctica la diferencia en la velocidad de ejecución entre el algoritmo de exponenciación binaria comparado con el algoritmo de exponenciación ingenua, sin optimizar. Empezaremos por cargar las funciones que necesitamos:

```
from cryptocalc import (  
    RFC_3526_SAFE_PRIME,  
    exp_mod,  
    naive_exp_mod  
)
```

Ahora fijaremos un número primo que utilizaremos como módulo y un valor base.

```
p = RFC_3526_SAFE_PRIME  
g = 2  
x = (g, p)  
print(f"Módulo primo elegido (p): {p}\n")  
print(f"Valor base elegido elegido (g): {g}\n")
```

Módulo primo elegido (p): 15525180923007089351309181312584817556313340494345143132023511949029

Valor base elegido elegido (g): 2

Para comparar los dos algoritmos, mediremos el tiempo que llevar elevar el punto base a la potencia $k = 100$ usando ambos métodos. Con exponenciación binaria:

```
import time

start = time.perf_counter()
exp_mod(x, 100)
end = time.perf_counter()

print(f"Elapsed time: {end - start:.6f} seconds")
```

Elapsed time: 0.000031 seconds

Usando exponenciación ingenua:

```
start = time.perf_counter()
naive_exp_mod(x, 100)
end = time.perf_counter()

print(f"Elapsed time: {end - start:.6f} seconds")
```

Elapsed time: 0.000038 seconds

Como podemos observar, no hay una diferencia muy significativa entre ambos algoritmos por el momento. Elevaremos ahora el punto base a la potencia $k = 100000$. Con exponenciación binaria:

```
start = time.perf_counter()
exp_mod(x, 100000)
end = time.perf_counter()

print(f"Elapsed time: {end - start:.6f} seconds")
```

Elapsed time: 0.000047 seconds

Usando exponenciación ingenua:

```
start = time.perf_counter()
naive_exp_mod(x, 100000)
end = time.perf_counter()

print(f"Elapsed time: {end - start:.6f} seconds")
```

Elapsed time: 0.010510 seconds

Podemos notar que en este caso ya existe una diferencia significativa entre ambos algoritmos. Repetiremos ahora este proceso tomando como potencia $k = 10000000$. Con exponenciación binaria:

```
start = time.perf_counter()
exp_mod(x, 10000000)
end = time.perf_counter()

print(f"Elapsed time: {end - start:.6f} seconds")
```

Elapsed time: 0.000104 seconds

Usando exponenciación ingenua:

```
start = time.perf_counter()
naive_exp_mod(x, 10000000)
end = time.perf_counter()

print(f"Elapsed time: {end - start:.6f} seconds")
```

Elapsed time: 1.077710 seconds

Aquí podemos notar como la diferencia entre ambos algoritmos se va haciendo más palpable. Finalmente, tomaremos como potencia $k = 1000000000$. Con exponenciación binaria:

```
start = time.perf_counter()
exp_mod(x, 1000000000)
end = time.perf_counter()

print(f"Elapsed time: {end - start:.6f} seconds")
```

Elapsed time: 0.000089 seconds

Usando exponenciación ingenua:

```
start = time.perf_counter()
naive_exp_mod(x, 1000000000)
end = time.perf_counter()

print(f"Elapsed time: {end - start:.6f} seconds")
```

Elapsed time: 109.516067 seconds

Aquí ya podemos notar un efecto mucho más significativo. Queda claro que si tomamos potencias mucho mayores el cálculo por exponenciación ingenua se vuelve rápidamente imposible de realizar en un tiempo razonable.

Sistema RSA para firmas digitales

En este notebook, mostraremos el proceso para generar las claves pública y privada, así como el proceso para generar una firma digital y su verificación correspondiente usando el sistema RSA. Empezaremos por cargar las funciones que necesitamos:

```
from cryptocalc import (
    rsa_key_generation,
    rsa_signature_generation,
    is_valid_rsa_signature,
    sha256_of_sentence,
)
```

Protocolo RSA para la generación de claves

Una vez que ya hemos cargado las librerías necesarias, empezamos por generar nuestras claves *pública y privada*:

```
(public_key, private_key) = rsa_key_generation(2048)
d = private_key[0]
e = public_key[0]
n = public_key[1]

print(f"Exponente público (e): {e}\n")
print(f"Exponente privado (d): {d}\n")
print(f"Módulo para las claves (n): {n}\n")
```

Exponente público (e): 65537

Exponente privado (d): 388295760787525036999011484783087264494748750242847265076878887257792266

Módulo para las claves (n): 5480528777966538528590488591783686182928595792790791293119852612196

Notemos que para la generación de estas claves utilizamos números primos generados de manera aleatoria de alrededor de 2048 bits de longitud. Se sigue que el módulo generado n tiene alrededor de 4096 bits de longitud. Es muy importante que el valor del exponente privado d no se divulge. En cambio, el exponente público e y el módulo n forman parte de la clave pública y son conocidos por todos los agentes involucrados, incluida Eva.

Protocolo RSA para la firma de mensajes

Para ilustrar el protocolo para la generación de firmas digitales, supongamos que Alicia desea firmar el mensaje 'Hello World!'. Empezaremos por generar el hash correspondiente:

```
m = 'Hello World!'
h = sha256_of_sentence(m)

print(f"Mensaje llano (m): {m}\n")
print(f"Hash del mensaje llano (h): {h}\n")
```

Mensaje llano (m): Hello World!

Hash del mensaje llano (h): 5767641308109300314800510755071958354011698523669642386092346649047

Con esta información, podemos generar ahora la firma correspondiente y agregarla al mensaje llano para generar el *mensaje firmado*:

```
s = rsa_signature_generation(private_key, h)
signed_message = (m, s)

print(f"El mensaje firmado es: {signed_message}\n")
```

El mensaje firmado es: ('Hello World!', 401640971359655275098729043939084183326020020732919847

Protocolo RSA para la verificación de firmas

Para verificar la firma, Beto aplica el protocolo RSA utilizando la clave pública de Alicia, el hash del mensaje y la firma correspondiente:

```
h = sha256_of_sentence(signed_message[0])
s = signed_message[1]

if is_valid_rsa_signature(public_key,h,s):
    verificacion_firma = "La firma es válida"
```

```
else:
    verificacion_firma = "La firma es inválida"

print(verificacion_firma)
```

La firma es válida

Protocolo Diffie-Hellman para la generación de una clave compartida

En este notebook, mostraremos el proceso de generar una clave compartida utilizando el protocolo Diffie-Hellman. Empezaremos por cargar las funciones que necesitamos:

```
from cryptocalc import (
    RFC_3526_SAFE_PRIME,
    RFC_3526_SOPHIE_PRIME,
    generate_private_key,
    diffie_hellman_public_key_generator,
    diffie_hellman_shared_key_generator
)
```

Para este ejemplo, utilizaremos uno de los primos seguros listados en la especificación [RFC 3526](#) y generaremos las claves secretas de Alicia y Beto de manera aleatoria:

```
p = RFC_3526_SAFE_PRIME
q = RFC_3526_SOPHIE_PRIME
g = 2
a = generate_private_key(q)
b = generate_private_key(q)

print(f"Primo Seguro (p): {p}\n")
print(f"Primo de Sophie Germain (q): {q}\n")
print(f"Generador (g): {g}\n")
print(f"Clave privada de Alicia (a): {a}\n")
print(f"Clave privada de Beto (b): {b}")
```

Primo Seguro (p): 1552518092300708935130918131258481755631334049434514313202351194902966239949

Primo de Sophie Germain (q): 77625904615035446756545906562924087781566702471725715660117559745

Generador (g): 2

Clave privada de Alicia (a): 63697899823199294312278252486684835957300921317408335372817339039

Clave privada de Beto (b): 6613655176058448749519916636176705570570186589138638552032214468889

Generamos ahora las claves públicas de Alicia y Beto:

```
A = diffie_hellman_public_key_generator(p, g, a)
B = diffie_hellman_public_key_generator(p, g, b)

print(f"Clave pública de Alicia (A): {A}\n")
print(f"Clave pública de Beto (B): {B}")
```

Clave pública de Alicia (A): 11670488272236588836129467431951491412487902670794676240235400489

Clave pública de Beto (B): 5333711045026768409841114830794551583170827230075921314899738097351

Finalmente, generamos las claves compartidas

```
s_a = diffie_hellman_shared_key_generator(p, B, a)
s_b = diffie_hellman_shared_key_generator(p, A, b)

print(f"Clave compartida generada por Alicia (s_a): {s_a}\n")
print(f"Clave compartida generada por Beto (s_b): {s_b}\n")
```

Clave compartida generada por Alicia (s_a): 12346645623250654247470746939489244060160751163725

Clave compartida generada por Beto (s_b): 1234664562325065424747074693948924406016075116372511

Como podemos observar, ambas claves son iguales, lo cual ilustra la corrección del protocolo.

Curvas Elípticas

En este notebook, mostraremos un ejemplo de una curva elíptica y las operaciones que se pueden realizar sobre sus puntos. Empezaremos por cargar las funciones que necesitamos:

```
from cryptocalc import EllipticCurve
```

Usaremos la curva elíptica:

$$y^2 = x^3 + 6x + 23$$

sobre el campo finito $\mathbb{F}_{103}$. Su discriminante y j -invariante son:

```
c1 = 6
c0 = 23
p = 103

curve = EllipticCurve(p, c1, c0)

print(f"Discriminante modulo p: {curve.get_discriminant()}")
print(f"j-invariante modulo p: {curve.j_invariant()}")
```

```
Discriminante modulo p: 7
j-invariante modulo p: 87
```

Como nuestro campo base es pequeño, podemos calcular todos sus puntos de manera explícita:

```
E_p = curve.points()
print("Lista de puntos sobre la curva:\n")
for i, point in enumerate(E_p, start=1):
    print(f"{i}: {point}")
```

Lista de puntos sobre la curva:

```
1: infy
2: (0, 34)
3: (0, 69)
4: (1, 37)
5: (1, 66)
6: (3, 45)
7: (3, 58)
8: (4, 27)
9: (4, 76)
10: (8, 45)
11: (8, 58)
12: (11, 9)
13: (11, 94)
14: (12, 22)
15: (12, 81)
16: (13, 49)
17: (13, 54)
18: (18, 35)
19: (18, 68)
20: (22, 20)
21: (22, 83)
```


22: (27, 35)
23: (27, 68)
24: (29, 22)
25: (29, 81)
26: (32, 34)
27: (32, 69)
28: (34, 44)
29: (34, 59)
30: (36, 37)
31: (36, 66)
32: (37, 4)
33: (37, 99)
34: (38, 11)
35: (38, 92)
36: (42, 10)
37: (42, 93)
38: (43, 13)
39: (43, 90)
40: (47, 43)
41: (47, 60)
42: (51, 25)
43: (51, 78)
44: (54, 18)
45: (54, 85)
46: (57, 40)
47: (57, 63)
48: (58, 35)
49: (58, 68)
50: (60, 17)
51: (60, 86)
52: (61, 7)
53: (61, 96)
54: (62, 22)
55: (62, 81)
56: (63, 40)
57: (63, 63)
58: (64, 2)
59: (64, 101)
60: (65, 50)
61: (65, 53)
62: (66, 37)
63: (66, 66)
64: (67, 4)
65: (67, 99)

66: (70, 12)
67: (70, 91)
68: (71, 34)
69: (71, 69)
70: (72, 15)
71: (72, 88)
72: (77, 25)
73: (77, 78)
74: (78, 25)
75: (78, 78)
76: (79, 28)
77: (79, 75)
78: (81, 26)
79: (81, 77)
80: (82, 3)
81: (82, 100)
82: (86, 40)
83: (86, 63)
84: (88, 36)
85: (88, 67)
86: (89, 39)
87: (89, 64)
88: (90, 23)
89: (90, 80)
90: (92, 45)
91: (92, 58)
92: (94, 8)
93: (94, 95)
94: (95, 9)
95: (95, 94)
96: (96, 16)
97: (96, 87)
98: (99, 48)
99: (99, 55)
100: (100, 9)
101: (100, 94)
102: (102, 4)
103: (102, 99)

Consideramos ahora dos puntos sobre la curva:

$$P = (95, 9) \quad \text{y} \quad Q = (79, 28)$$

Y realizamos los siguientes cálculos:

```
P = (95, 9)
Q = (79, 28)
R = curve.addition(P,Q)
S = curve.double(P)
T = curve.inverse(P)

print(f"P + Q = {R}")
print(f"2P = {S}")
print(f"-P = {T}")
```

```
P + Q = (66, 66)
2P = (34, 44)
-P = (95, 94)
```

Como aplicación a estas operaciones en curvas elípticas, consideraremos el problema de guardar los hash de ciertos passwords. Para este ejemplo, dado un password m calcularemos mP como su hash:

```
list_of_passwords = [20, 27, 72]
list_of_hashed_passwords = [curve.multiplication(m, P) for m in list_of_passwords]

print("")
print("Contraseñas originales:")
print(list_of_passwords)
print("\nHashes almacenados:")
print(list_of_hashed_passwords)
print("")
```

```
Contraseñas originales:
[20, 27, 72]
```

```
Hashes almacenados:
[(89, 64), (67, 99), (32, 34)]
```

Protocolo Diffie-Hellman de Curvas Elípticas (ECDH) para la generación de una clave compartida

En este notebook, mostraremos el proceso de generar una clave compartida utilizando el protocolo ECDH. Empezaremos por cargar las funciones que necesitamos:

```
from cryptocalc import (  
    EllipticCurve,  
    SECP256K1,  
    generate_elliptic_private_key,  
)
```

Para este ejemplo, utilizaremos la curva **secp256k1** la cual se encuentra especificada en el documento [SEC 2: Recommended Elliptic Curve Domain Parameters](#) y generaremos las claves secretas de Alicia y Beto de manera aleatoria:

```
p = SECP256K1["p"]  
c1 = SECP256K1["a"]  
c0 = SECP256K1["b"]  
G = SECP256K1["G"]  
n = SECP256K1["n"]  
h = SECP256K1["h"]  
curve = EllipticCurve(p,c1,c0)  
a = generate_elliptic_private_key(n)  
b = generate_elliptic_private_key(n)  
  
print(f"Primo Base (p): {p}\n")  
print(f"Coeficiente del término lineal: {c1}\n")  
print(f"Coeficiente del término constante: {c0}\n")  
print(f"Generador (G): {G}\n")  
print(f"Orden del Generador (n): {n}\n")  
print(f"Clave privada de Alicia (a): {a}\n")  
print(f"Clave privada de Beto (b): {b}")
```

Primo Base (p): 115792089237316195423570985008687907853269984665640564039457584007908834671663

Coeficiente del término lineal: 0

Coeficiente del término constante: 7

Generador (G): (55066263022277343669578718895168534326250603453777594175500187360389116729240,

Orden del Generador (n): 115792089237316195423570985008687907852837564279074904382605163141518

Clave privada de Alicia (a): 11412362292333573509766425743200603104620249080105178516015638385

Clave privada de Beto (b): 4557343802234857857689462518440899950102602524994888391932499450395

Generamos ahora las claves públicas de Alicia y Beto:

```
A = curve.multiplication(a,G)
B = curve.multiplication(b,G)

print(f"Clave pública de Alicia (A): {A}\n")
print(f"Clave pública de Beto (B): {B}")
```

Clave pública de Alicia (A): (1499094746543689566485154487537075048571682464064914190464437494

Clave pública de Beto (B): (511068250429316559251148446044452547144347862368745318490380719652

Finalmente, generamos las claves compartidas

```
S_a = curve.multiplication(a,B)
S_b = curve.multiplication(b,A)

print(f"Clave compartida generada por Alicia: {S_a}\n")
print(f"Clave compartida generada por Beto: {S_b}\n")
```

Clave compartida generada por Alicia: (3664219430569436609331136726352843371452880743197612345

Clave compartida generada por Beto: (366421943056943660933113672635284337145288074319761234527

Como podemos observar, ambas claves son iguales, lo cual ilustra la corrección del protocolo.